SE-315 Cloud Computing • Semester Project

Lakehouse Architecture with Apache Iceberg, Spark and Trino using CDC Ingestion

Group 04: Aliyan Zulfiqar (leader) and Sehrosh Riaz

NUST Balochistan Campus (NBC), Department of Computer Science



Problem statement

1

Analytics on live data

Analysts need current OLTP data, but heavy queries must not load the operational database.

2

Batch ETL is stale

Nightly ETL leaves reports out of date and copies whole tables on every run.

3

What we need

Fresh, correct and cheap-to-query history: stream only the changes, keep every version.

Our answer: capture every committed change with CDC and merge it into open Iceberg tables that any SQL engine can read.

Goals and evaluation metrics

Goal	How we measure it
CDC freshness	Commit → visible in Trino (p50 / p95)
Ingestion throughput	Rows/s initial load, records/s per batch
Query latency	8 queries, median of 5 runs, before/after
Small files and storage	File counts, avg file size, MinIO bytes
Correctness	Row counts + checksum() vs PostgreSQL

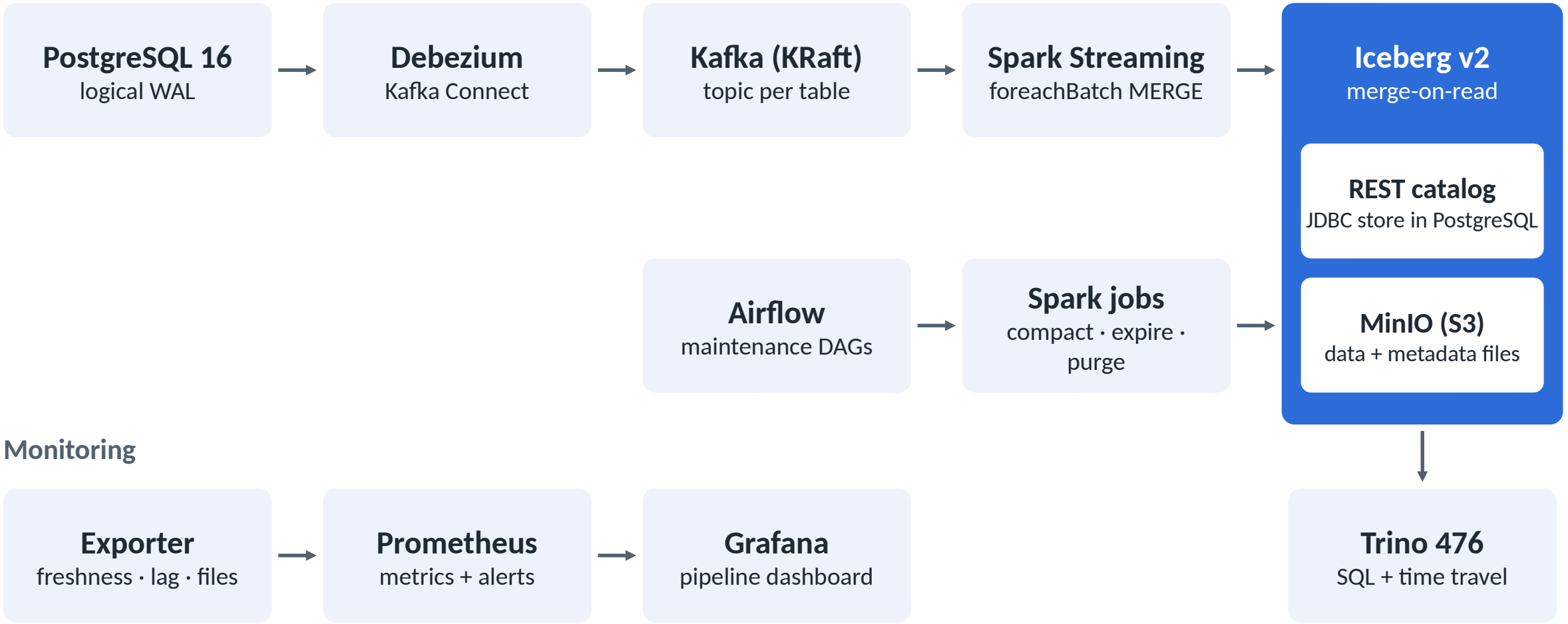
Datasets

TPC-H sf1
1,650,030 rows: orders 1.5M, customer 150k, nation, region

Synthetic OLTP workload
Continuous inserts, updates and deletes against live tables

Source: README.md (evaluation metrics), results/ingest_load.json

Solution architecture



CDC and ingestion design

- Logical replication with the pgoutput plugin (wal_level=logical)
- Debezium snapshots existing rows, then streams every change to one topic per table: pg.shop.<table>
- Iceberg format v2, merge-on-read: updates write small delete files instead of rewriting data files
- Kafka checkpoint + idempotent MERGE = effectively-once results

Each micro-batch (foreachBatch), per table

1. **Schema discovery:** read the schema in the event, create or evolve the table
2. **Dedupe by key:** keep only the latest change per primary key
3. **One MERGE INTO:** deletes, updates and inserts in a single commit
4. **Commit snapshot:** retried with backoff if compaction commits first

Source: docs/architecture.md, src/topologies/spark/cdc_to_iceberg.py

Schema evolution and time travel

Schema evolution (automatic)

- ALTER TABLE … ADD COLUMN in PostgreSQL becomes a new Iceberg column
- int → bigint widening is applied in place
- No restart and no data rewrite
- Old rows read NULL for the new column (schema-on-read)

Time travel (every batch is a snapshot)

```
SELECT * FROM orders
  FOR VERSION AS OF <id>;
SELECT * FROM orders
  FOR TIMESTAMP AS OF <ts>;
```

- $snapshots lists every commit
- Change audit: current version EXCEPT an older one

Source: scripts/demos/schema_evolution.sh, scripts/demos/time_travel.sh

Optimisation with Airflow + Spark

orders: partition evolution to months(o_orderdate), sorted by o_orderdate, o_custkey

customer: sorted by c_nationkey, c_custkey

Layout + compact

43.9 s

Apply partition spec and sort order, sorted rewrite of data files

→

Rewrite deletes

22.8 s

Rewrite position-delete files and manifests

→

Expire snapshots

23.1 s

Drop old snapshots and the files only they used

→

Remove orphans

67.7 s

Delete unreferenced files (24 h minimum age)

DAG iceberg_layout_optimization wall time: **169.8 s** · an hourly iceberg_maintenance DAG does routine compaction

Source: results/benchmark-20261007-181123/summary.md (compaction overhead), docs/architecture.md

GDPR erasure (extra credit)

1

DELETE in PostgreSQL

CDC writes a position-delete file. The row disappears from the current snapshot.

2

Still reachable

Older snapshots still point at the original data file, so time travel returns the row.

3

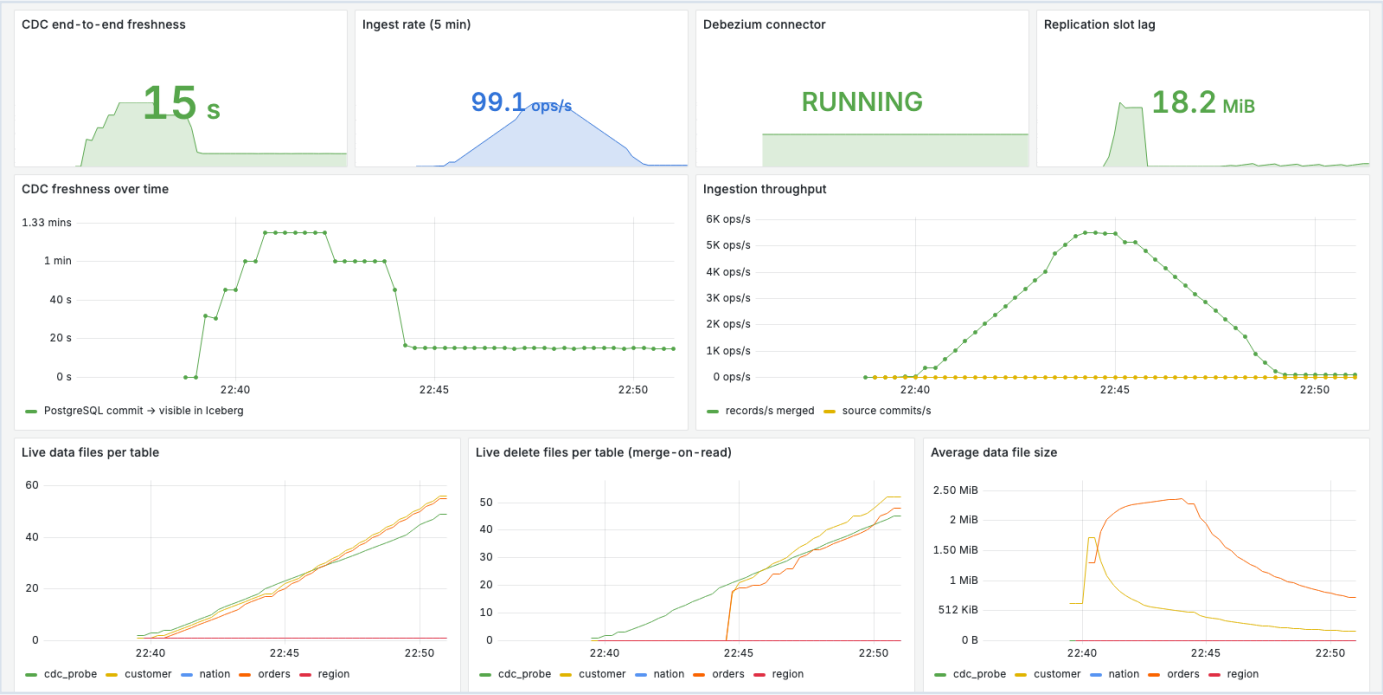
Purge DAG

iceberg_gdpr_purge rewrites the data files and expires history. The data is physically gone.

A merge-on-read delete hides a row; only rewriting the files and expiring the snapshots erases it.

Source: scripts/demos/gdpr_delete.sh, DAG iceberg_gdpr_purge

Monitoring and alerting



Grafana "Lakehouse CDC pipeline" dashboard (16 panels) fed by a custom exporter

Prometheus alert rules

CdcFreshnessHigh
freshness > 120 s for 2 min

DebeziumConnectorDown
connector not RUNNING

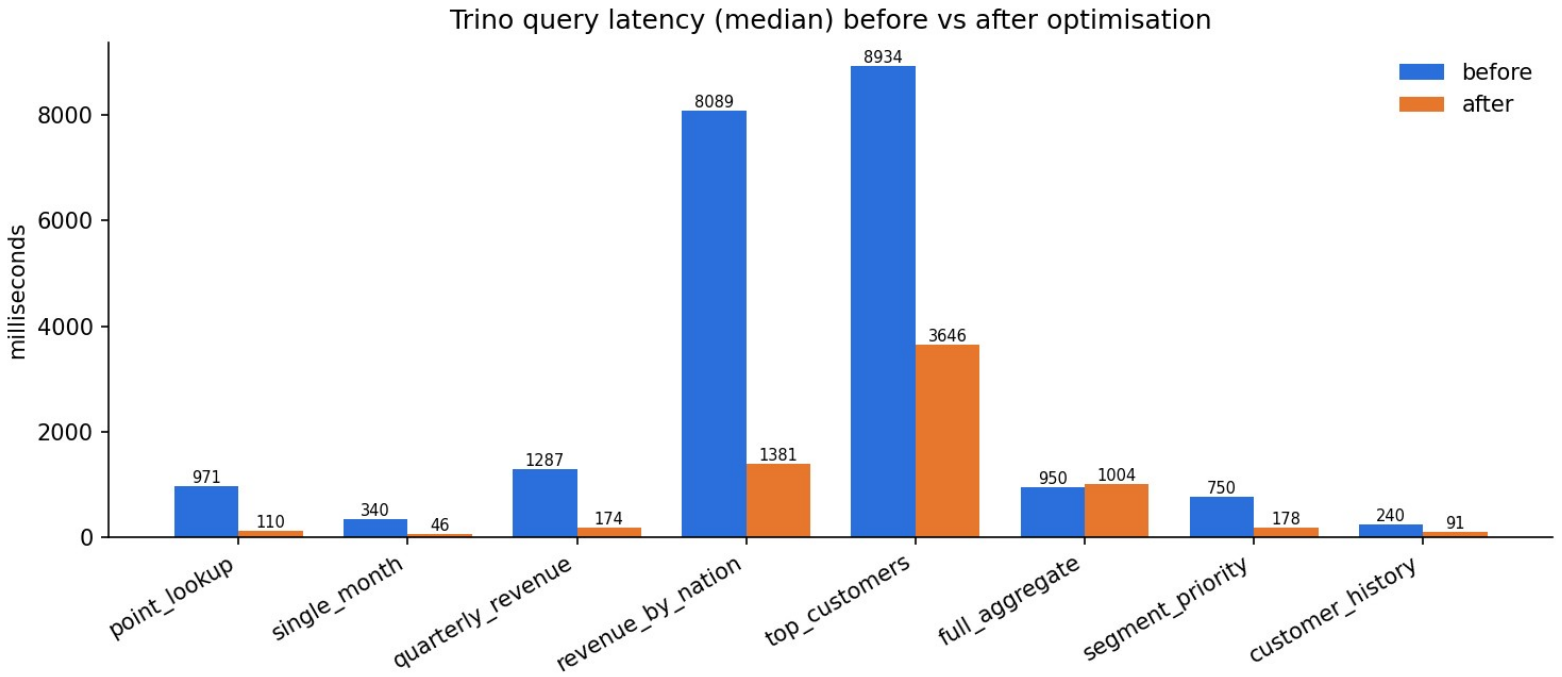
ReplicationSlotLagHigh
retained WAL > 512 MB

SmallFilesAccumulating
> 500 data files, avg < 8 MB

StreamingStalled
no batch for 10 min, changes pending

Source: docs/report/grafana_dashboard.png, config/monitoring/alerts.yml

Results: query latency



Full-table aggregate (q06) is flat at 0.95×: it reads every row either way, so there is nothing to prune.

3.25×

faster overall: 21.6 s → 6.6 s for the 8 queries

8.81×

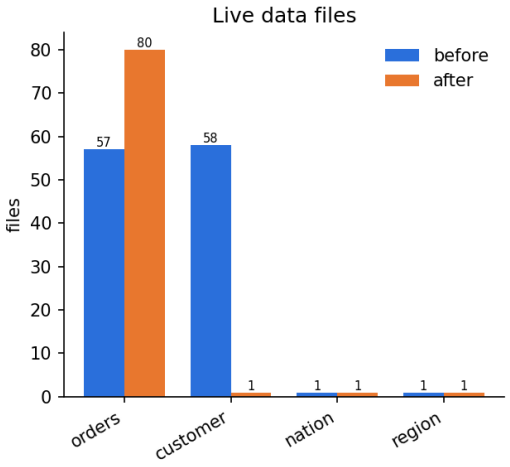
point lookup: 971 → 110 ms

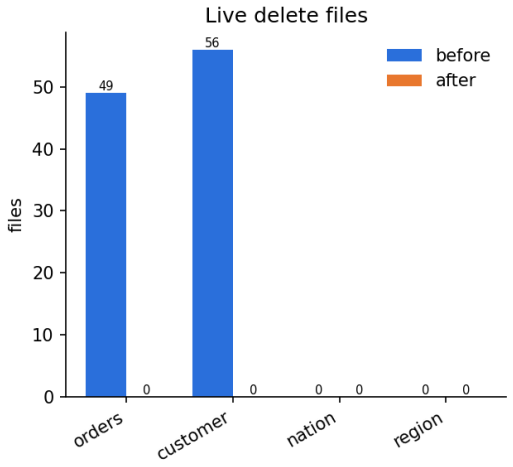
19,488

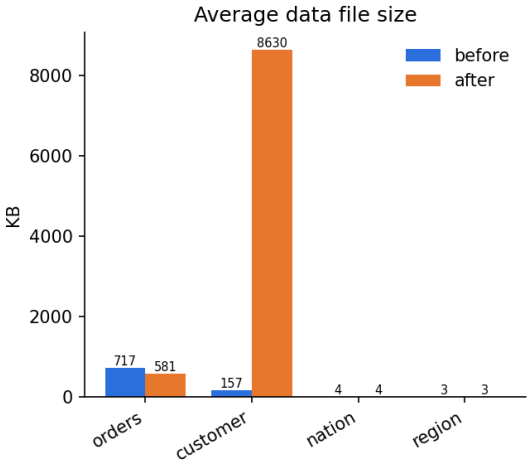
rows scanned for one month, down from 1,513,192

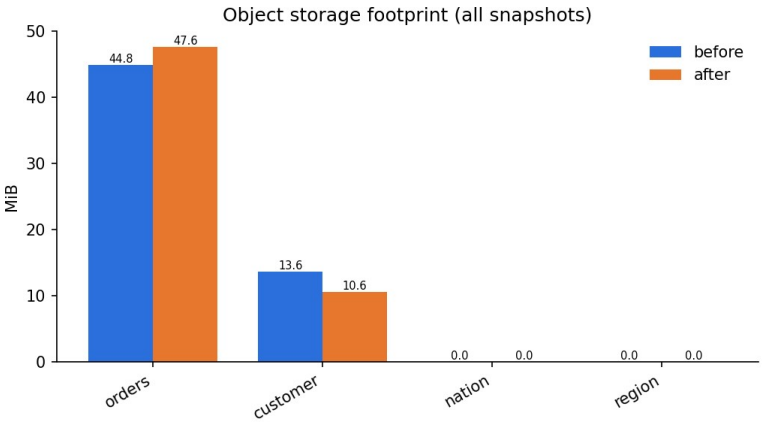
Source: results/benchmark-20261007-181123/summary.md (Trino, median of 5 runs)

Results: small files and storage









Trade-off: orders went from 57 to 80 files and +6.2% storage (44.8 → 47.6 MB), because monthly partitions split it into smaller files.

105 → 0

live delete files (orders 49, customer 56)

57 → 1

orders snapshots after expiry (customer 58 → 1)

−22.4%

customer storage: 58 small files → 1

Source: results/benchmark-20261007-181123/summary.md

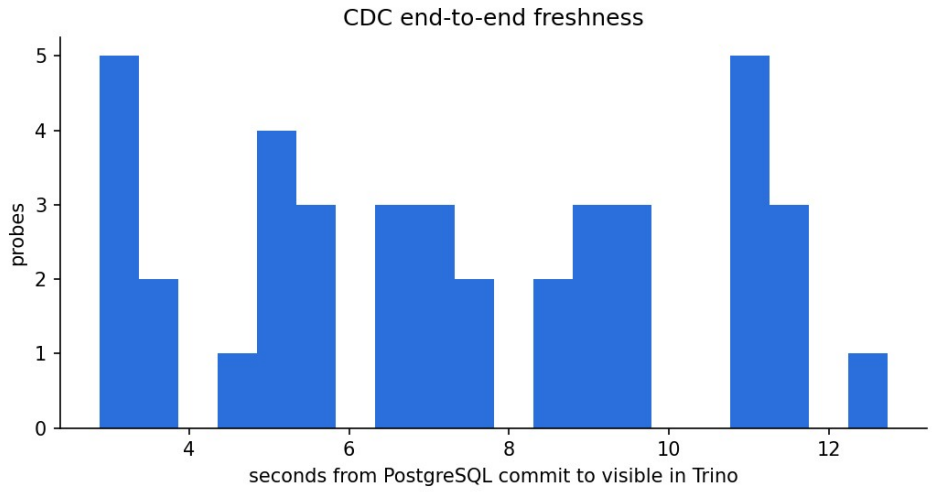
Results: ingestion, freshness, correctness

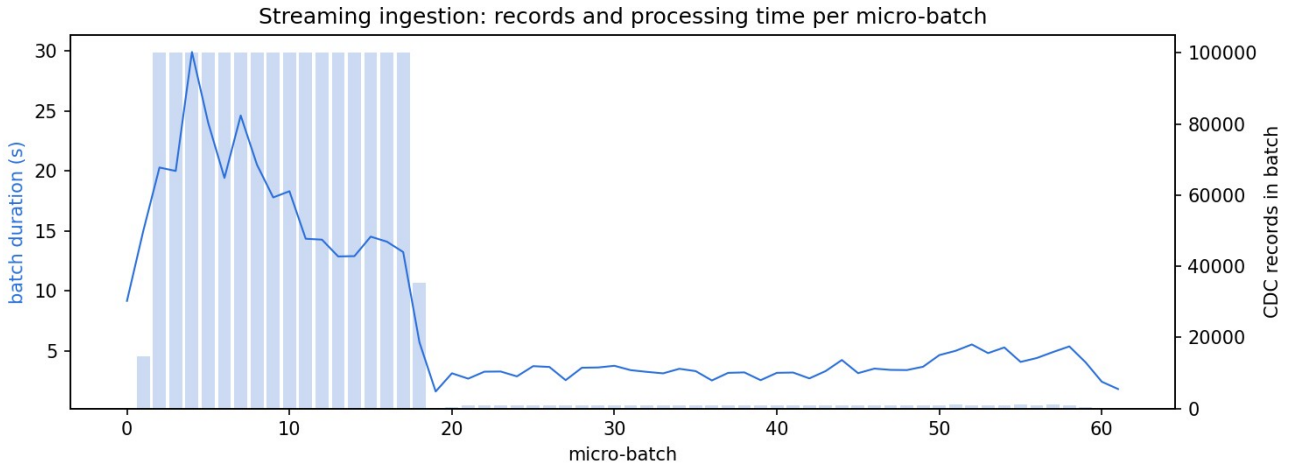
4,843.5
rows/s initial load (1,650,030 rows in 340.7 s)

3,574.9
records/s streaming MERGE (peak batch 7,751.9)

7.3 s
freshness p50 (p95 11.73 s, 40/40 probes)

4 / 4
tables: row counts and checksums match





Source: results/benchmark-20261007-181123/summary.md, results/ingest_load.json, results/freshness.json

Technical challenges

Challenge	Fix
SQLite REST catalog: SQLITE_BUSY under concurrent commits	Catalog moved to PostgreSQL (JDBC)
Spark MERGE fails with identifier fields (Iceberg 1.9)	Primary key kept as a table property
remove_orphan_files needs a Hadoop S3 filesystem	S3 listing via file_list_view (boto3)
Trino's 432 MB per-node memory too small for joins	Raised memory limits + spill-to-disk
Sorted rewrite of 1.5M rows: heap OOM in 1 GB driver	Resized job and container memory
Debezium JSON carries its schema (~3 KB/message)	zstd in Kafka; schema read by prefix
MinIO official Docker images withdrawn (2025)	Maintained community build
Whole stack must fit in Docker's 8 GB	Per-service limits; local-mode Spark

Source: docs/slides/SLIDES_BRIEF.md (technical challenges), docs/architecture.md

Conclusion and future work

What we showed

- A complete CDC lakehouse running in Docker on one laptop
- Fresh: 7.3 s median from commit to query
- Correct: checksums match on all 4 tables
- Optimised: 3.25× faster query suite, 105 delete files → 0

Future work

1. **Hudi vs Iceberg:** same workload, compare table formats
2. **Apache Atlas:** data lineage across the pipeline
3. **Kubernetes / AKS:** cloud deployment of the stack
4. **Avro + Schema Registry:** smaller events than JSON
5. **Apache Flink:** streaming engine alternative to Spark

Thank you. Questions?